

AFRILANG Stdlib

std/c/graphbipart

Bibliothèque AFRILANG `std/c/graphbipart` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : isBipartite,

```
`import "std/c/graphbipart" . `use graphbipart`
```

- `isBipartite(adj list number, n number) ? bool`
- `bipartiteColors(adj list number, n number) ? list number`
- `partitionSizes(adj list number, n number) ? list number`
- `crossEdgeCount(adj list number, n number) ? number`
- `sameSideEdges(adj list number, n number) ? number`
- `bipartiteDensity(adj list number, n number) ? number`
- `hasOddCycle(adj list number, n number) ? bool`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphbipart` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : `isBipartite`,

```
`import "std/c/graphbipart"` . `use graphbipart`  
- `isBipartite(adj list number, n number) ? bool`  
- `bipartiteColors(adj list number, n number) ? list number`  
- `partitionSizes(adj list number, n number) ? list number`  
- `crossEdgeCount(adj list number, n number) ? number`  
- `sameSideEdges(adj list number, n number) ? number`  
- `bipartiteDensity(adj list number, n number) ? number`  
- `hasOddCycle(adj list number, n number) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphbipart"  
use graphbipart
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? isBipartite(adj list number, n number) ? bool  
? bipartiteColors(adj list number, n number) ? list  
? partitionSizes(adj list number, n number) ? list  
? crossEdgeCount(adj list number, n number) ? number  
? sameSideEdges(adj list number, n number) ? number  
? bipartiteDensity(adj list number, n number) ? number  
? hasOddCycle(adj list number, n number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/graphbipart"  
use graphbipart  
  
create result = isBipartite(/* args */)   
say result  
  
create result = bipartiteColors(/* args */)   
say result  
  
create result = partitionSizes(/* args */)   
say result  
  
create result = crossEdgeCount(/* args */)   
say result  
  
create result = sameSideEdges(/* args */)   
say result  
  
create result = bipartiteDensity(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/graphbipart"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module graphbipart
```

```
function isBipartite(adj list number, n number) returns bool
end

function bipartiteColors(adj list number, n number) returns list number
end

function partitionSizes(adj list number, n number) returns list number
end

function crossEdgeCount(adj list number, n number) returns number
end

function sameSideEdges(adj list number, n number) returns number
end

function bipartiteDensity(adj list number, n number) returns number
end

function hasOddCycle(adj list number, n number) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphbipart` (tier complex, category complex). Exports 7 function(s): isBipartite, bipartiteColors

```
`import "std/c/graphbipart" . `use graphbipart`  
- `isBipartite(adj list number, n number) ? bool`  
- `bipartiteColors(adj list number, n number) ? list number`  
- `partitionSizes(adj list number, n number) ? list number`  
- `crossEdgeCount(adj list number, n number) ? number`  
- `sameSideEdges(adj list number, n number) ? number`  
- `bipartiteDensity(adj list number, n number) ? number`  
- `hasOddCycle(adj list number, n number) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphbipart"  
use graphbipart
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? isBipartite(adj list number, n number) ? bool  
? bipartiteColors(adj list number, n number) ? list  
? partitionSizes(adj list number, n number) ? list  
? crossEdgeCount(adj list number, n number) ? number  
? sameSideEdges(adj list number, n number) ? number  
? bipartiteDensity(adj list number, n number) ? number  
? hasOddCycle(adj list number, n number) ? bool
```

4. Usage examples

```
import "std/c/graphbipart"  
use graphbipart  
  
create result = isBipartite(/* args */)   
say result  
  
create result = bipartiteColors(/* args */)   
say result  
  
create result = partitionSizes(/* args */)   
say result  
  
create result = crossEdgeCount(/* args */)   
say result  
  
create result = sameSideEdges(/* args */)   
say result  
  
create result = bipartiteDensity(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphbipart"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphbipart
```

```
function isBipartite(adj list number, n number) returns bool
end

function bipartiteColors(adj list number, n number) returns list number
end

function partitionSizes(adj list number, n number) returns list number
end

function crossEdgeCount(adj list number, n number) returns number
end

function sameSideEdges(adj list number, n number) returns number
end

function bipartiteDensity(adj list number, n number) returns number
end

function hasOddCycle(adj list number, n number) returns bool
end
end
```